LangConfig on WSL2 (Ubuntu) — native PostgreSQL setup

已在 WSL2 + Ubuntu 上以**原生** PostgreSQL 16（不使用 Docker）验证通过。 如需 Docker 方式或 macOS，请参阅 `docs/TEAM_INSTALL.md`。

What you end up with

服务	地址
前端（Vite/React）	http://localhost:1421
后端 API（FastAPI）	http://localhost:8765
Swagger / OpenAPI	http://localhost:8765/docs
PostgreSQL（原生）	`localhost:5432` — 用户 `langconfig`，数据库 `langconfig`
Neo4j（可选）	http://localhost:7474（浏览器） · `bolt://localhost:7687`

本指南假定的版本：**Python 3.13、Node ≥ 20.19 或 ≥ 22.12、PostgreSQL 16 + pgvector**。

Contents

Part 1 — install (do once)

1. Base tools · 2. Clone · 3. PostgreSQL 16 + pgvector · 4. Create the role + database · 5. `.env` ·
2. Python dependencies · 7. Seed the database · 8. Verify · 9. Frontend dependencies · 10. Start everything ·
3. Optional — Neo4j · 12. Troubleshooting · 13. Daily workflow

Part 2 — WSL platform notes 14. WSL tuning · 15. Windows ↔ WSL integration · 16. Docker on WSL · 17. Disk space and the WSL VHDX · 18. Backups and restore · 19. Starting the stack automatically · 20. Playwright browsers · 21. SSH keys for git

> ⚠️ **请把仓库放在 Linux 文件系统里**（`~/langconfig`），**绝不要**放在 Windows 盘符 （`/mnt/c/...`）。`uv` 使用硬链接与 mmap，而 drvfs/9p 文件系统会在 `uv sync` 期间 产生 `bus error`（SIGBUS）。磁盘预算：虚拟环境约 2 GB，uv 缓存可达数 GB。

1. Base tools

```
sudo apt update && sudo apt install -y git curl ca-certificates unzip lsof
lsb_release -cs          # 'noble' = 24.04  |  'jammy' = 22.04
```

`start.sh` 会用 `lsof` 清理占用 8765/1421 端口的残留进程。

2. Clone

```
cd ~
git clone git@github.com:sailor723/langconfig.git langconfig
# no SSH key on this WSL? use HTTPS instead:
#   git clone https://github.com/sailor723/langconfig.git langconfig

cd langconfig
git log --oneline -1        # expect the current main tip
git status -sb              # expect: ## main...origin/main  (no ahead/behind)
```

3. PostgreSQL 16 + pgvector

Ubuntu 24.04 (`noble`) — packages are in the distro repos

```
sudo apt install -y postgresql-16 postgresql-16-pgvector
```

Ubuntu 22.04 (`jammy`) — needs the official PGDG repo first

Jammy 自带 PostgreSQL 14，且没有 pgvector 软件包，因此需要先添加 PGDG：

```
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc --fail \
    https://www.postgresql.org/media/keys/ACCC4CF8.asc
sudo sh -c 'echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.asc] \
    https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > \
    /etc/apt/sources.list.d/pgdg.list'
sudo apt update
sudo apt install -y postgresql-16 postgresql-16-pgvector
```

快速判断：执行 `apt-cache policy postgresql-16 postgresql-16-pgvector` —— 如果两者都显示 真实的 `Candidate:`，说明这台机器不需要 PGDG 这一步。

Start the service and verify

```
sudo service postgresql start
pg_isready -h localhost -p 5432      # → localhost:5432 - accepting connections
pg_lsclusters                        # → "16  main  5432  online"
```

WSL 不会自动启动 PostgreSQL。 每次重启 WSL 之后都要再执行一次 `sudo service postgresql start`（如果你在 `/etc/wsl.conf` 里启用了 systemd， 也可以用 `sudo systemctl start postgresql`）。

⚠️ 如果 `pg_lsclusters` 已经显示一个 **14** 集群占用 5432 端口，它会和 PostgreSQL 16 争抢该端口。

4. Create the role + database (once)

```
sudo -u postgres psql -c "CREATE ROLE langconfig LOGIN PASSWORD 'langconfig_dev' CREATEDB SUPERUSER;"
sudo -u postgres psql -c "CREATE DATABASE langconfig OWNER langconfig;"
```

这两个权限都必不可少：

- `CREATEDB` —— 初始化脚本要创建 `chinook` 示例数据库（步骤 10）
- `SUPERUSER` —— `CREATE EXTENSION vector` 是**以应用用户身份**执行的，而 pgvector 不是受信任扩展。缺少该权限会报错：`permission denied to create extension "vector"`。（若让 `seed_all.py` 自动创建角色，它只会授予 `CREATEDB`，所以请按上面的命令手动创建。）

验证：

```
PGPASSWORD=langconfig_dev psql -h localhost -p 5432 -U langconfig -d langconfig -c "select version();"
```

5. Create `.env` at the repo root

整个项目只有**一个** `.env`，位于 `~/langconfig/.env`（后端通过 `../../.env` 读取）。该文件已被 gitignore 忽略。

```
cd ~/langconfig
cp env_example .env
chmod 600 .env

# required: 64-hex Fernet master key for API keys stored in the DB
sed -i "s|^API_KEY_ENCRYPTION_KEY=.*|API_KEY_ENCRYPTION_KEY=$(openssl rand -hex 32)|" .env
```

原生安装只需关注这几项：

变量	取值	说明
`POSTGRES_HOST`	`localhost`	
`POSTGRES_PORT`	**`5432`**	**5432 = 原生，5433 = Docker。** 这是最常见的出错点
`POSTGRES_DB` / `POSTGRES_USER` / `POSTGRES_PASSWORD`	`langconfig` / `langconfig` / `langconfig_dev`	
`API_KEY_ENCRYPTION_KEY`	64 位十六进制	请在界面上保存任何 API Key **之前**设置好：之后再改会导致已存密钥无法解密
`LANGCONFIG_WORKSPACE_ROOT`	`backend/.workspace_data`	项目级 `AGENTS.md` 目录 —— **不是** 每个 agent 的工作目录

不要设置 `DATABASE_URL` —— 它会覆盖整个 `POSTGRES_*` 配置块。

确认应用实际解析出的连接串：

```
cd ~/langconfig/backend
uv run python -c "from dotenv import load_dotenv; load_dotenv('../.env'); from db.db_url import
        compose_db_url; print(compose_db_url())"
# → postgresql://langconfig:langconfig_dev@localhost:5432/langconfig
```

6. Python dependencies

```
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.local/bin/env          # or restart the shell

cd ~/langconfig/backend
uv sync                          # creates backend/.venv (Python 3.13) — resolves 282 packages
uv run python -c "import fastapi, langgraph, deepagents, sqlalchemy, psycopg, neo4j, openpyxl;
        print('deps OK')"
```

`uv sync` 会打印 `Resolved 282 packages`，并且只安装**增量部分** —— `Installed N packages` 那一行不是总数。要查看真实总数：`uv run python -c "import importlib.metadata as m; print(len(list(m.distributions())))"` → 约 260。 如果出现 SIGBUS：`rm -rf .venv && uv cache clean && UV_LINK_MODE=copy uv sync` （原因：仓库位于 `/mnt/c`、磁盘写满，或缓存损坏 —— 见本文件顶部的提示以及 §12 中的 `bus error` 一行）。

7. Seed the database

```
cd ~/langconfig/backend
WORKSPACE_ROOT="$HOME/langconfig/backend/outputs" uv run python scripts/seeds/seed_all.py
```

`WORKSPACE_ROOT` 请填你自己的真实路径（绝不要用 macOS 的 `/Users/...` 路径）。它是可选的 —— 只用于设定身份回填（identity backfill）写入 agent 提示词中的目录，和 `.env` 里的 `LANGCONFIG_WORKSPACE_ROOT` **不是**同一个东西。该脚本是幂等的；只有需要强制重新导入 tools/agents 数据时才加 `--replace`。

10 个步骤：初始化服务器 → 初始化数据库（+pgvector）→ alembic stamp head → settings → 28 个模型 → 6 个搜索后端 → tools → deep agents → 身份回填（步骤 8.5）→ skills → chinook。

`[1/10] init_server` 应该输出 *"role 'langconfig' already exists"* / *"database 'langconfig' already exists"* —— 这说明 §4 已完成。如果它输出 *"Cannot connect as a superuser"*，说明角色/数据库从未创建成功。

8. Verify

```
cd ~/langconfig
bash scripts/wsl_check.sh
```

全部通过的基线：48 张表 · `alembic_version = 029_add_tool_definition` · 已启用 `vector 0.8.2` · `settings_documents` 7 · `models` 28 · `custom_tools` 67 · `deep_agent_templates` 51 · `skills` 37 · chinook 11 张表。

（该脚本是只读的；它输出的 ❌ 行就是待办清单。）

9. Frontend dependencies

Vite 7 要求 **Node ≥ 20.19 或 ≥ 22.12**（`engines: ^20.19.0 || >=22.12.0`）。 Ubuntu apt 源里的 `nodejs` 往往太旧 —— 请使用 nvm：

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
source ~/.nvm/nvm.sh
nvm install 22
node -v                          # v22.x

cd ~/langconfig
npm install                      # REQUIRED — nothing is shipped in git
```

每次 `git pull` 后只要 `package.json` / `package-lock.json` 有变化，就要重新执行 `npm install`； 否则 vite 会因为缺少模块而启动失败。

10. Start everything

```
cd ~/langconfig
sudo service postgresql start    # WSL does not auto-start Postgres
bash start.sh                    # db check + backend + frontend
```

预期输出：

```
── PostgreSQL ── ✅ PostgreSQL already running on :5432
── Backend ──── ✅ Backend healthy on http://localhost:8765
── Frontend ─── ✅ Frontend running on http://localhost:1421
```

其他用法：`bash start.sh --be`（仅后端） · `bash start.sh --fe`（仅前端）。

停止全部服务：

```
kill $(cat /tmp/langconfig-be.pid) $(cat /tmp/langconfig-fe.pid)
```

验证两个端口（vite 使用 `strictPort: true`，因此 1421 被占用时它会直接退出，而不会换到别的端口）：

```
ss -ltnp | grep -E ':(8765|1421)'
curl -s -o /dev/null -w "backend  -> %{http_code}\n" http://127.0.0.1:8765/api/chat/health
curl -s -o /dev/null -w "frontend -> %{http_code}\n" http://localhost:1421/
```

请打开 http://localhost:1421 —— 在 Windows 里也同样适用（WSL2 会转发 `localhost`）。 请使用 `localhost`，**不要**用 WSL 的 IP（`172.x.x.x`）：vite 的 `allowedHosts` 只允许 `localhost`， 用 IP 会得到 `Blocked request. This host is not allowed.`

后端本身也提供 `GET /`（纯 JSON 信息）和 `/docs`（Swagger）—— 不启动前端时可以用来检查 API， 但真正的界面只由 1421 上的 vite 提供。

> 注意： `start.sh` 是把前端放到后台启动的，因此 vite 启动失败并不会让脚本中断 —— 你只会看到 1421 "connection refused"。要看到 vite 的真实报错，请在前台运行： `cd ~/langconfig && npm run dev`。
>
> 某些版本里，当 PostgreSQL 服务尚未启动时， `start.sh` 会用 `brew`（macOS 命令）去启动它， 这在 WSL 上会直接中断脚本。按上面的做法先启动 Postgres 就能完全避开这条路径。

11. Optional — Neo4j (Cypher agent / Graph Studio)

不使用 Neo4j 时应用一切正常；只有这两个功能需要它。 （需要 WSL 里的 Docker，或者把 Neo4j 跑在别的主机上并把 `NEO4J_URI` 指过去。）

```
docker run -d --name langconfig-neo4j \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/password \
  neo4j:2025.10.1
```

`.env` 已经指向 `bolt://localhost:7687` / `http://localhost:7474`。

导入 HR 知识图谱（社区版兼容的 JSON seed —— 无需许可证）：

```
cd ~/langconfig/backend
uv run python scripts/seeds/neo4j-hr/import_hr_seed.py bolt://localhost:7687 neo4j password \
    scripts/seeds/neo4j-hr/hr_nodes.json scripts/seeds/neo4j-hr/hr_rels.json
# → VERIFY: nodes=6978 rels=23281
```

> 同一目录下的 `neo4j.dump` 是**企业版**转储（block 存储格式）—— 没有企业版许可证无法恢复。 请使用上面的 JSON seed。

12. Troubleshooting

现象	原因	解决
8765 上 `connection refused`	后端未运行 / 崩溃	`cd backend && .venv/bin/uvicorn main:app --port 8765` 查看堆栈
1421 上 `connection refused`	从未执行 `npm install`，或 vite 报错	`cd ~/langconfig && npm run dev`（前台）读取报错
`Vite requires Node.js version 20.19+`	apt 安装的 Node 过旧	`nvm install 22`
`Error: Port 1421 is already in use`	残留的 vite（strictPort 会退出）	`ss -ltnp | grep 1421` → 杀掉该 PID
`Blocked request. This host is not allowed.`	用了 WSL 的 IP 而不是 localhost	改用 `http://localhost:1421`，或设置 `VITE_ALLOWED_HOSTS=*`
`brew: command not found` in `start.sh`	仅适用于 macOS 的分支，且 Postgres 已停止	`sudo service postgresql start` 后重试
`permission denied to create extension "vector"`	角色缺少 `SUPERUSER`	`sudo -u postgres psql -c "ALTER ROLE langconfig SUPERUSER;"`
`extension "vector" is not available`	未安装 pgvector 软件包	安装 `postgresql-16-pgvector`
`could not connect to server ... :5432`	Postgres 已停止（WSL 重启）	`sudo service postgresql start`
`could not connect ... :5433`	`.env` 仍是 Docker 端口	设置 `POSTGRES_PORT=5432`

现象	原因	解决
`role "langconfig" already exists`	无害	seed 是幂等的
`ModuleNotFoundError: sqlalchemy`	虚拟环境缺失/过期，或目录不对	`rm -rf backend/.venv && cd backend && uv sync`
`ModuleNotFoundError: neo4j`	早于声明该依赖的版本	`git pull && cd backend && uv sync`
`bus error` during `uv sync`	仓库在 `/mnt/c`、磁盘写满，或缓存损坏	移到 `~/`、`df -h ~`、`uv cache clean`
`-bash: you: No such file or directory`	粘贴了 macOS 的 `WORKSPACE_ROOT=/Users/<you>/...`	改用 `$HOME/langconfig/backend/outputs`
seed step 2 fails on extensions	创建角色时没有 `SUPERUSER`	见 §4

常用诊断命令：

```
cd ~/langconfig/backend
uv run python -c "import importlib.metadata as m; print('installed:', len(list(m.distributions())))"
uv run python -c "from dotenv import load_dotenv; load_dotenv('../.env'); from db.db_url import
        compose_db_url; print(compose_db_url())"
bash ../scripts/health_check.sh                 # postgres + API + seeded data (needs backend
        running)
```

13. Daily workflow

```
cd ~/langconfig
sudo service postgresql start     # after any WSL restart
bash start.sh                     # → :8765 API, :1421 UI
# ...
kill $(cat /tmp/langconfig-be.pid) $(cat /tmp/langconfig-fe.pid)
```

每次 `git pull` 之后：如果 `backend/pyproject.toml` / `uv.lock` 有变化就重新执行 `uv sync`； 如果 `package.json` / `package-lock.json` 有变化就重新执行 `npm install`，然后重启服务。

Part 2 — WSL platform notes

以下内容均为 WSL 特有（普通 Ubuntu 服务器或 macOS 不需要）。

14. WSL tuning (`/etc/wsl.conf`, `.wslconfig`)

启用 systemd（从而拥有 `systemctl`，并让服务可以自行启动）：

```
sudo tee /etc/wsl.conf >/dev/null <<'EOF'
[boot]
systemd=true

[interop]
enabled=true

[automount]
options = "metadata,umask=22,fmask=11"
EOF
```

然后**在 Windows 中**（PowerShell）执行：`wsl --shutdown`，再重新打开终端。 验证：`systemctl is-system-running` → `running`（或 `degraded`）。

限制 WSL 的内存/CPU：在 Windows 的 `C:\Users\<you>\.wslconfig` 中设置 —— WSL2 默认会占用 最多 50% 的内存，这一点很关键，因为 `uv sync` 需要构建重量级 wheel（torch、spacy、onnxruntime）：

```
[wsl2]
memory=8GB
processors=4
swap=2GB
localhostForwarding=true      # keep true: Windows -> WSL localhost works
```

用 `wsl --shutdown` 使其生效（所有发行版都会重启）。

其他能避免问题的 WSL 习惯：

```
git config --global core.autocrlf input     # avoid CRLF in shell scripts
sudo hwclock -s                             # fix clock drift (breaks TLS/HTTPS clones)
```

15. Windows ↔ WSL integration

任务	做法
在 Windows 中打开界面	`http://localhost:1421` —— WSL2 会转发 `localhost`（需要 `localhostForwarding=true`）
用资源管理器打开仓库	在仓库内执行 `explorer.exe .`，或访问 `\\wsl$\Ubuntu\home\<user>\langconfig`
用 VS Code 编辑	安装 **WSL** 扩展，然后在仓库内执行 `code .`
向终端粘贴	Windows Terminal：`Ctrl+Shift+V` 或右键
首次启动后端	Windows 防火墙可能弹窗 —— 允许**专用网络**

⚠ **不要**通过 `/mnt/c` 或 `\\wsl$\...` 让 Windows 工具去改写仓库里的文件（会造成 行尾/属性反复变动）。请把工作副本放在 `~/` 下。

默认情况下，**其他设备**（手机、局域网）无法访问本应用：vite 只允许 `localhost`， 你需要 WSL 的 IP 再加一条防火墙规则。如果确实需要，请用 `VITE_ALLOWED_HOSTS=*` 启动 vite 并做端口转发。

16. Docker on WSL (only for the optional Neo4j, §11)

方案 A —— Docker Desktop： 在 Windows 上安装，然后 *Settings → Resources → WSL integration → 启用你的 Ubuntu 发行版*。之后在 WSL 中 `docker ps` 即可用。

方案 B —— 直接在 WSL 里装引擎（不用 Desktop）：

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER      # then close/reopen the terminal
sudo service docker start          # after each WSL restart (or enable systemd)
docker ps                          # verify
```

然后运行 §11 中的 Neo4j 命令。

17. Disk space and the WSL VHDX

容易让人意外的体积（在可用安装上实测）：

目录	大小
`backend/.venv`	约 2 GB
`~/.cache/uv`	最高可达数 GB
`node_modules`	约 0.5 GB
Postgres 数据 + `backend/data` 上传文件	随使用增长

```
df -h ~                                                  # free space
du -sh ~/.cache/uv backend/.venv node_modules 2>/dev/null   # biggest consumers

# reclaim
uv cache clean                # ~/.cache/uv (re-downloads next sync)
npm cache clean --force
sudo apt clean
docker system prune -af       # if you use Docker
```

WSL 的磁盘镜像（VHDX）**只会增长** —— 在 WSL 内删除文件并不会让它收缩。要在 Windows 上回收空间：

```
wsl --shutdown
wsl --manage Ubuntu --set-sparse true     # WSL 2.0+ (distro name from: wsl -l -v)
```

传统做法：`diskpart` → `select vdisk file="…\ext4.vhdx"` → `attach vdisk readonly` → `compact vdisk` → `detach vdisk`。

> 经验法则：在 `uv sync` 之前，确保 WSL 内**至少剩余 8–10 GB** 空间；空间耗尽正是安装期间 出现 `bus error` 的原因之一。

18. Backups and restore

`.env` 中保存着 **Fernet 主密钥**，请单独备份。一旦丢失，数据库中已存储的所有 API Key 都将无法解密。

```
# 1. secrets + config
cp ~/langconfig/.env ~/langconfig.env.bak          # or copy to /mnt/c/Users/<you>/

# 2. database (custom-format dump: compressed + restorable)
pg_dump -h localhost -p 5432 -U langconfig -Fc langconfig > ~/langconfig-$(date +%F).dump

# 3. restore (fresh machine or after a mistake)
createdb -h localhost -p 5432 -U langconfig langconfig
pg_restore -h localhost -p 5432 -U langconfig -d langconfig --clean --if-exists ~/langconfig-YYYY-MM-
        DD.dump
```

如果你使用了它们，也建议一并复制：`~/langconfig/backend/data/`（上传的文档）和 `~/langconfig/backend/outputs/`（生成的产物）。两者都不在 git 中。没有转储也可以只重建表结构：`seed_all.py` + `alembic upgrade head`。

19. Starting the stack automatically

已启用 systemd（§14） —— 让 Postgres 自己管理，应用可选：

```
sudo systemctl enable postgresql
```

```
# /etc/systemd/system/langconfig.service
[Unit]
Description=LangConfig (backend :8765 + frontend :1421)
After=network-online.target postgresql.service
Wants=postgresql.service

[Service]
Type=oneshot
RemainAfterExit=yes
User=%i
WorkingDirectory=/home/%i/langconfig
ExecStart=/usr/bin/bash -lc 'cd /home/%i/langconfig && bash start.sh'
ExecStop=/usr/bin/bash -lc 'kill $(cat /tmp/langconfig-be.pid) $(cat /tmp/langconfig-fe.pid)'

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable --now langconfig@$USER.service     # note: %i = your username
```

这里用 `Type=oneshot` 是有意为之：`start.sh` 会把 uvicorn/vite 放到后台然后退出， 若使用普通的 service 类型会被判定为失败。

没有 systemd 时 —— 使用 Windows 任务计划程序，触发器设为*登录时*：

```
wsl.exe -d Ubuntu -u <user> bash -lc "cd ~/langconfig && bash start.sh"
```

注意：WSL 只在有终端/发行版存活时运行；执行 `wsl --shutdown` 后服务就会停止（若需要 7×24 常驻， 请部署到真正的 Linux 主机）。

20. Playwright browsers (optional tools)

`playwright` 是项目依赖，但它的浏览器需要单独下载。只有浏览器自动化相关工具才需要：

```
cd ~/langconfig/backend
uv run playwright install chromium
# missing system libraries?
uv run playwright install-deps chromium     # uses sudo
```

21. SSH keys for git

```
ssh-keygen -t ed25519 -C "wsl-langconfig"     # Enter to accept defaults
cat ~/.ssh/id_ed25519.pub                     # paste into GitHub → Settings → SSH keys
ssh -T git@github.com                         # → "Hi <user>! You've successfully authenticated"
```

没有 SSH？请改用 HTTPS + 个人访问令牌（见 §2）。